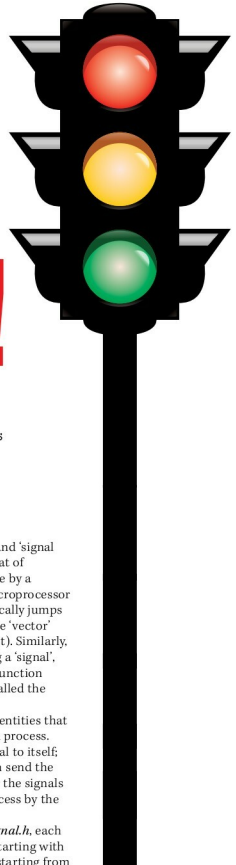


Watch Out for the Signals!



What in the world is the 'signals' framework and how can systems programmers make use of it?

*W*e say people are clever when they understand the 'signals' in real life. The case is the same with Linux too. The right signals sent across at the right time in the system make it fast and responsive. This article throws light on this 'signals' framework in the Linux system, and explores how systems programmers can make use of it.

The framework

The motivation behind this framework of 'signals' is to make the process aware that something has happened in the system, and the target process should perform some predefined set of actions to keep the system running smoothly. These actions range from 'self-termination' to 'clean-up'.

The concept of 'signals' and 'signal handling' is analogous to that of the 'interrupt' handling done by a microprocessor. When a microprocessor receives an interrupt, it typically jumps to a fixed location (called the 'vector' location for a given interrupt). Similarly, in Linux, a process receiving a 'signal', typically invokes a specific function registered with the signal, called the 'handler' for a given signal.

There could be multiple entities that can send a 'signal' to a given process. The process can send a signal to itself; other running processes can send the signals to a given process or the signals could be sent to a given process by the Linux kernel too.

As defined in *include/signal.h*, each signal has a specific name starting with 'SIG' and a unique number starting from